

Checkpoint de Avance Generador YAPar

Período del reporte: 3-17 de mayo de 2026 (Sprint 1 y Sprint 2)

Repositorio: <https://github.com/lfmendoza/compiladores-yapar>

Fecha del reporte: 17 de mayo de 2026

1. OKR de la fase de compilación

Objetivo: Entregar la fase de compilación YAPar con autómata LR(0) canónico, tabla SLR(1) y motor del parser funcionando sobre el conjunto de gramáticas de prueba acordadas, con documentación final completa antes de la fecha límite.

KR	Resultado clave	Meta	Estado	% avance
KR1	Arquitectura documentada y aprobada.	1 doc	Completo	100 %
KR2	Módulos del pipeline con tests al 80 % de cobertura mínima.	8 / 8	Completo	100 %
KR3	Parser ejecuta sobre la batería de gramáticas de prueba.	3 / 3	En curso	67 %
KR4	Documentación final, video y entrega antes de la fecha límite.	1 entrega	En curso	50 %

Avance OKR global: aproximadamente 80 %.

2. Resumen de los sprints

Sprint 1 (3–10 de mayo de 2026)

El sprint 1 se dedicó a la fase de diseño arquitectónico y a la infraestructura del proyecto. Se completó el documento de arquitectura (KR1) junto con el diagrama de módulos en draw.io. Se estableció la infraestructura de CI con GitHub Actions, se definió el contrato Token(tipo, lexema, línea, columna) que conecta el YALex previo con el pipeline YAPar, y se realizó un spike de viabilidad con Graphviz. Se iniciaron las estructuras de datos centrales del módulo 3 y el esqueleto del lector .yalp.

Sprint 2 (13–17 de mayo de 2026)

El sprint 2 cerró la implementación completa del pipeline. En orden de dependencia se terminaron el módulo 1 (lector DSL), módulo 2 (puente YALex), módulo 3 (estructuras de gramática con augmentación), módulo 4 (cómputo de FIRST/FOLLOW por punto fijo), módulo 5 (colección canónica LR(0) con deduplicación por frozenset), módulo 6 (tabla SLR(1) con reporte de conflictos), módulo 7 (motor LR de pila con árbol de parse) y módulo

8 (renderizador Graphviz). La suite de tests unitarios supera el 80 % de cobertura sobre src/yapar/.

3. Tablero Kanban

El tablero refleja el estado del proyecto al cierre del sprint 2. Las columnas Backlog, En progreso, Revisión y Hecho agrupan las tarjetas por estado. Cada tarjeta corresponde a un módulo o entregable del OKR y tiene un responsable asignado explícito.

Columna	Tarjetas
Hecho	Arquitectura (KR1), CI/CD, scaffolding, fixtures YALex, módulos 1–8, suite de tests unitarios
En progreso	Documento de entrega final (KR4), demo end-to-end
Revisión	—
Backlog	Video de demostración

4. Distribución de responsabilidades

Integrante	Módulos asignados	Foco del sprint	Entregable
Fernando M.	1, 2, 3	Lector .yalp, puente YALex, estructuras de gramática	PR #4 (reader), PR #5 (grammar)
Fernando M.	4, 5	FIRST/FOLLOW y autómata LR(0)	PR #7 (sets + automaton)
Fernando M.	6, 7, 8	Tabla SLR(1), motor LR de pila y visualizador	PR #9 (table + engine + viz)

5. Changelog de commits

Extracto de git log del repositorio durante el período del reporte. Se siguen las convenciones de Conventional Commits (feat, fix, docs, chore, test) para que el changelog pueda generarse automáticamente al cierre del proyecto.

Hash	Autor	Fecha	Mensaje
a244d1e	Fernando Mendoza	2026-05-07	feat(analysis): yapar compiler analysis

6. Riesgos identificados y mitigaciones

Riesgo	Impacto / Probabilidad	Mitigación	Estado
Gramáticas de prueba con conflictos shift/reduce no resolubles por SLR.	Alto / Media	El módulo 6 reporta cada conflicto con el ítem y el lookahead. Si aparecen, se simplifica la gramática o se escala a LALR(1).	Controlado
Bug en la deduplicación de estados rompe la terminación del closure/goto.	Alto / Baja	El _index frozenset→id se prueba con un caso patológico (gramática con ϵ -producciones). Test de regresión obligatorio en el módulo 5.	Controlado
Desincronización con el formato de tokens del YALex previo.	Medio / Media	Contrato Token(tipo, lexema, línea, columna) definido en arquitectura. Test de integración con outputs reales de YALex.	Controlado
Acumulación de trabajo al final del semestre por solapamiento con otros cursos.	Alto / Alta	Sprints de una semana con checkpoint público los viernes. Si un sprint cierra con más de 2 tarjetas atrasadas, se renegocia el alcance.	Abierto

7. Arquitectura del sistema

El pipeline YAPar transforma un archivo .yalp más la definición léxica exportada por YALex en una tabla SLR(1) y un motor de parser capaz de aceptar o rechazar cadenas de entrada. La figura siguiente muestra el diagrama de módulos.

[Imagen no disponible: arquitectura_yapar.png]

Módulo	Archivo	Responsabilidad
1	reader.py	Tokenizador y parser del DSL .yalp

2	lexer_bridge.py	Adaptador YALex → Token stream. Contrato Token(tipo, lexema, linea, columna). Único acoplamiento con YALex.
3	grammar.py	Symbol, Production, Grammar (frozen dataclasses) + augmentación S'
4	sets.py	Cómputo de FIRST y FOLLOW mediante iteración hasta punto fijo
5	automaton.py	Colección canónica LR(0): closure, goto, deduplicación por frozenset
6	table.py	Tabla SLR(1) ACTION/GOTO con reporte de conflictos shift/reduce
7	engine.py	Motor LR de pila con árbol de parse como resultado
8	viz.py	Renderizador Graphviz del autómata LR(0) — escribe .dot o PNG

Contrato de integración

El único punto de acoplamiento entre YALex y YAPar es la clase Token, definida en lexer_bridge.py como una NamedTuple con los campos tipo, lexema, linea y columna. El centinela de fin de cadena es Token('\$', '\$', -1, -1). El módulo lexer_bridge.py es el único componente que conoce el formato de serialización del DFA exportado por YALex (pickle o JSON según la versión), manteniendo el resto del pipeline desacoplado.

8. Próximos pasos y plan de cierre

Con el pipeline completo, las tareas pendientes son la validación sobre la batería de gramáticas de prueba (KR3), la redacción del documento de entrega final y la grabación del video de demostración (KR4).

Día	Tarea	Responsable	Criterio de aceptación
Lun 13 may	Cerrar módulo 1 (lector .yalp) y abrir PR.	Fernando M.	PR + tests
Mar 14 may	Cerrar módulo 3 (gramática + augmentación).	Fernando M.	merge a main
Mié 15 may	Módulos 4–5	Fernando M.	spike 4 h

	(FIRST/FOLLOW + autómata LR(0)).		
Jue 16 may	Módulos 6-7 (tabla SLR + motor).	Fernando M.	rama feat/slr- engine
Vie 17 may	Módulo 8 (viz) + demo interna end- to-end.	Fernando M.	demo OK
Vie 17 may	Entrega checkpoint de avance.	Fernando M.	documento subido